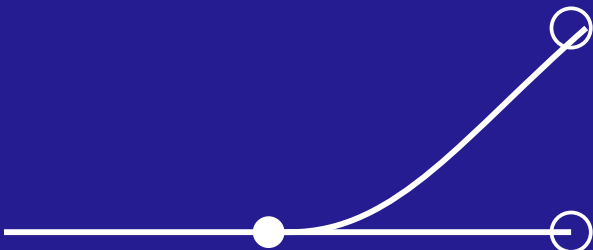


Pilotage d'un carrousel de conditionnement

De la première logique de séquençement à une logique TwinCAT structurée et documentée.

Chater Bach-char

Automaticien



Architecture en modes, cycle automatique et validation par essais.

Objet

Présenter l'état actuel du carrousel en cohérence avec les objets TwinCAT du dépôt et distinguer clairement l'implémenté des évolutions.

Périmètre

Structured Text · huit bacs
indexation pneumatique · réinitialisations et défauts

Entreprise

TQS · Tri Qualité Service

Date

6 septembre 2026

Sommaire

01 · Démarche : Contexte et objectif	3
02 · Réalisation : Système et périmètre implémenté	4
03 · Architecture TwinCAT : Modes et flux de données	5
04 · Séquencement : Machine d'états du cycle AUTO	6
05 · Structured Text : Comptage, bacs et indexation	7
06 · Robustesse : Réinitialisations et gestion des défauts	8
07 · Essais TwinCAT : Relevé de validation fonctionnelle	9
08 · Retour d'essais : Cas limites détectés et corrigés	10
09 · Périmètre maîtrisé : Limites et évolutions possibles	11
10 · Synthèse : Conclusion	12
Annexe A · TwinCAT 3 : Types et interface du bloc	13
Annexe B · Initiative personnelle : IHM ajoutées après l'entretien	14
Annexe B · Suite : TwinCAT HMI V2	15

01 à 02

contexte, système étudié et périmètre réellement implémenté;

03 à 06

architecture TwinCAT, machine d'états et traitements ST;

07 à 08

relevé des essais fonctionnels et cas limites corrigés;

09 à 10

limites, évolutions et synthèse du travail réalisé;

Annexes

types et interface PLC, puis IHM réalisées après l'entretien.

01 · DÉMARCHÉ

Contexte et objectif

Conserver la trace de la première approche, puis décrire fidèlement l'état actuel du projet TwinCAT.

Positionnement

Lors du test, une version volontairement simplifiée a été développée afin de présenter rapidement le principe de fonctionnement. Le projet a ensuite été repris sous TwinCAT pour approfondir l'architecture, la gestion des huit bacs, les réinitialisations et plusieurs cas limites.

Objectif du rapport

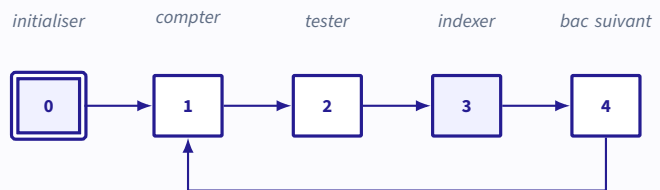
Le document distingue la **solution de principe montrée pendant l'entretien** de la **version approfondie actuellement présente dans le dépôt**. Les résultats d'essais déjà consignés sont conservés comme tels; la présente mise à jour vérifie statiquement les objets TwinCAT et n'affirme pas avoir rejoué le runtime. Les fonctions non réalisées sont identifiées comme limites ou évolutions.

Fil conducteur

1. poser le cycle nominal;
2. isoler les modes et les états;
3. traduire la séquence en ST;
4. provoquer des cas limites;
5. recouper le relevé d'essais avec les sources PLC.

Première lecture fonctionnelle

Le schéma de séquençement initial reste une trace utile : il montre le raisonnement présenté à l'automaticien sans présenter cette première version comme aussi complète que le projet final.



Ce qui a changé ensuite

La reprise sous TwinCAT remplace cette séquence linéaire par deux niveaux : un mode global INIT / AUTO / DEFAULT, puis une machine d'états dédiée au cycle AUTO.

8

bacs mémorisés

4

cycles / index

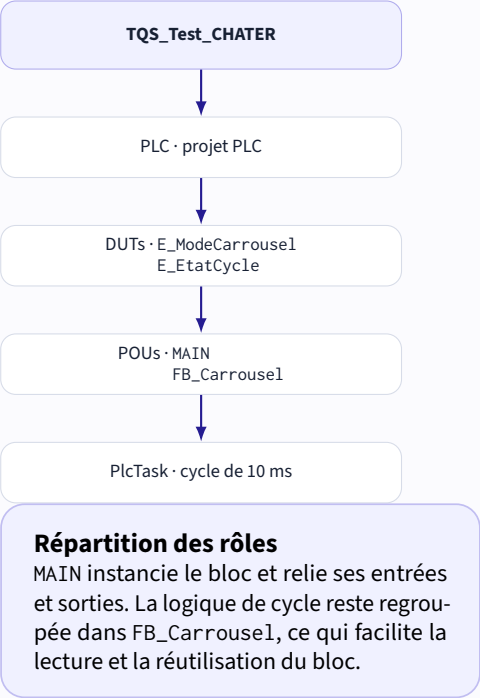
10 s

appui long

Système et périmètre implémenté

Un bloc fonctionnel unique porte la logique métier ; le programme principal reste volontairement léger.

Structure du projet XAE



Fonctions présentes dans la version réalisée

Fonction	Comportement implémenté
Référencement	attente de l'entrée logique bZero, puis affectation du bac 1
Comptage	front montant du signal du capteur laser avec R_TRIG
Mémoire	huit compteurs dans aPieceBac[1..8]
Indexation	quatre cycles logiques complets : commande avant, puis commande arrière
Temporisation	durée de chaque phase simulée par un TON réglé avec tVerin
Bouclage	retour logique du bac 8 vers le bac 1
Bac suivant déjà plein	attente de l'opérateur dans ATTENTE_BAC jusqu'à la réinitialisation du bac
Réinitialisation	appui bref : bac concerné; maintien 10 s : huit compteurs
Défaut	paramètres ou états invalides, commandes à FALSE, reprise par INIT

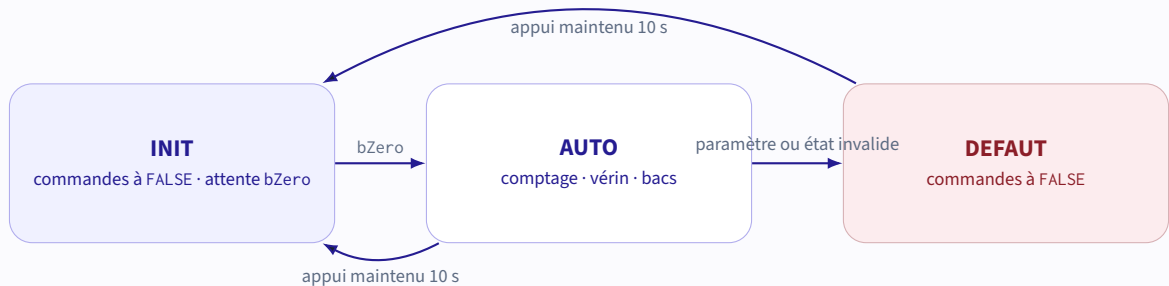
Interprétation retenue

Le terme « 4 impulsions » a été traduit par quatre cycles de commande complets : bVerinAv, puis bVerinAr, chaque phase étant validée par temporisation avant l'incrément de nImp. Aucun retour de fin de course n'est câblé dans le bloc.

03 · ARCHITECTURE TWINCAT

Modes et flux de données

Trois modes structurent le fonctionnement de la machine; une seconde machine d'états détaille le cycle automatique.



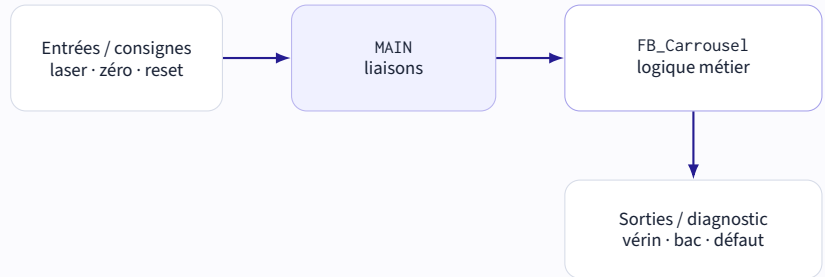
DUTs retenus

E_ModeCarrousel
INIT AUTO DEFAUT

E_EtatCycle
ATTENTE, COMPTAGE, BAC_PLEIN,
VERIN_SORTIE, VERIN_RENTREE,
BAC_SUIVANT, ATTENTE_BAC.

Priorité globale
L'appui maintenu pendant 10 s sur l'un des huit boutons est traité avant le CASE des modes. Il remet les compteurs et les commandes à zéro, acquitte le défaut, puis impose un retour par INIT.

Flux du programme

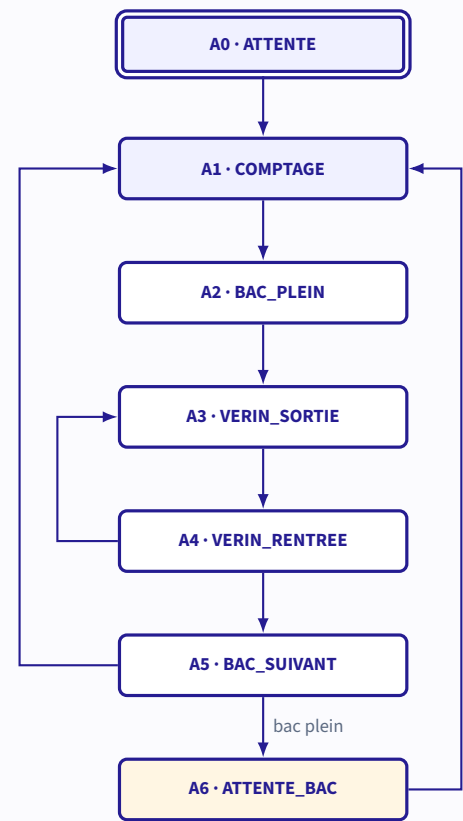


Règles de fonctionnement

- la logique de bac n'est exécutée qu'en AUTO;
- l'index nBac est contrôlé avant accès au tableau;
- les deux sorties de commande bVerinAv et bVerinAr ne sont jamais vraies simultanément;
- le changement de bac intervient après le quatrième cycle complet;
- un bac déjà plein bloque le cycle jusqu'à sa réinitialisation.

Machine d'états du cycle AUTO

La représentation reprend les états et les transitions réellement présents dans le bloc fonctionnel.



Transitions principales

Transition	Condition
A0 → A1	premier traitement de ATTENTE en mode AUTO; transition sans condition
A1 → A2	aPieceBac[nBac] >= nPieceBac
A2 → A3	transition immédiate
A3 → A4	temporisation de sortie écoulée
A4 → A3	temporisation de rentrée écoulée et nImp < 4
A4 → A5	temporisation écoulée et nImp >= 4
A5 → A1	nouveau bac sous la consigne
A5 → A6	nouveau bac déjà plein
A6 → A1	aResetBac[nBac] = TRUE

Lecture du cycle vérin

Une unité de nImp correspond à une phase avant temporisée suivie d’une phase arrière temporisée. Le numéro logique de bac ne change qu’après quatre cycles complets; le code ne vérifie pas la position mécanique du plateau.

Référencement

INIT exploite uniquement l’entrée logique bZero. La nature physique du capteur et la stratégie mécanique de recherche du zéro ne sont pas définies dans les sources; bZero = TRUE valide le référencement.

05 · STRUCTURED TEXT

Comptage, bacs et indexation

Des traitements courts, directement observables en ligne dans le bloc fonctionnel.

Front du capteur laser

```
1 fbLaser(CLK := bLaser);
2
3 IF fbLaser.Q
4 AND (NOT aResetBac[nBac]) THEN
5   aPieceBac[nBac] :=
6     aPieceBac[nBac] + 1;
7 END_IF
```

R_TRIG évite d'incrémenter plusieurs fois la même pièce lorsque le signal reste actif pendant plusieurs cycles PLC. Le même test est exécuté dans ATTENTE, avant le passage à COMPTAGE, puis dans COMPTAGE. La condition sur aResetBac[nBac] donne la priorité à la réinitialisation si les deux événements surviennent dans le même cycle. Le trigger ne remplace pas un filtrage matériel ou un traitement antiparasite. Le comptage reste limité à ces états : aucune pièce ne doit être présentée avant la validation du zéro ni pendant l'indexation.

Données par bac

aPieceBac : ARRAY[1..8] OF INT
Une valeur est conservée pour chaque bac. Seule la case correspondant à nBac est incrémentée.

Noms retenus

bLaser, bZero, nBac, nImp, tVerin et aResetBac restent courts et lisibles pour une mise au point en atelier.

Commandes pneumatiques temporisées

```
1 E_EtatCycle.VERIN_SORTIE:
2   bVerinAv := TRUE;
3   bVerinAr := FALSE;
4   fbTempoVerin(IN := TRUE,
5               PT := tVerin);
6
7   IF fbTempoVerin.Q THEN
8     bVerinAv := FALSE;
9     fbTempoVerin(IN := FALSE);
10    eEtatCycle :=
11      E_EtatCycle.VERIN_RENTREE;
12  END_IF
13
14 E_EtatCycle.VERIN_RENTREE:
15   bVerinAv := FALSE;
16   bVerinAr := TRUE;
17   fbTempoVerin(IN := TRUE,
18               PT := tVerin);
19
20   IF fbTempoVerin.Q THEN
21     bVerinAr := FALSE;
22     fbTempoVerin(IN := FALSE);
23     nImp := nImp + 1;
24
25     IF nImp >= 4 THEN
26       eEtatCycle :=
27         E_EtatCycle.BAC_SUIVANT;
28     ELSE
29       eEtatCycle :=
30         E_EtatCycle.VERIN_SORTIE;
31   END_IF
32 END_IF
```

Commandes déterministes

Au début de chaque cycle PLC, les deux sorties sont placées à FALSE. Seul l'état actif commande ensuite bVerinAv ou bVerinAr. En l'absence d'entrées de fin de course, le TON simule la durée de chaque mouvement; la fin de la phase arrière incrémente nImp.

Réinitialisations et gestion des défauts

Les commandes sont neutralisées avant toute reprise ; l'appui long sert également d'acquiescement.

Réinitialisations

```
1 bResetGlobal := FALSE;
2
3 FOR i := 1 TO 8 DO
4   IF aResetBac[i] THEN
5     aPieceBac[i] := 0; // reset bac
6   END_IF
7
8   aTempoReset[i](
9     IN := aResetBac[i],
10    PT := T#10S);
11
12   IF aTempoReset[i].Q THEN
13     bResetGlobal := TRUE;
14   END_IF
15 END_FOR
```

Effet de l'appui long
Le maintien pendant 10 s de l'un des huit boutons remet à zéro les huit compteurs, les deux commandes et nImp. Le défaut est acquitté, le temporisateur du vérin est réinitialisé et le mode repasse à INIT.

Effet individuel
Dès qu'un bouton aResetBac[i] est vrai, le compteur du bac i est maintenu à zéro. Si le bouton reste vrai pendant 10 s, le même signal déclenche ensuite la remise à zéro globale.

Conditions de défaut présentes

Condition	Réaction
nPieceBac <= 0	défaut immédiat, temporisateur arrêté et commandes neutralisées
tVerin <= T#0MS	défaut immédiat, temporisateur arrêté et commandes neutralisées
nBac hors de l'intervalle 1 à 8	blocage avant accès au tableau et passage en DEFAULT
état cycle incohérent	commandes à FALSE et défaut immédiat
mode incohérent	passage contrôlé en DEFAULT
mode DEFAULT	commandes à FALSE, temporisateur réinitialisé
appui de 10 s sur un bouton	remise à zéro globale, acquiescement et retour par INIT



Portée réelle
Cette version gère des incohérences logicielles et de paramétrage. Elle ne contrôle aucun fin de course, ne réalise pas de diagnostic mécanique détaillé et ne constitue pas une fonction de sécurité machine.

Relevé de validation fonctionnelle

Les résultats ci-dessous reprennent les essais TwinCAT déjà consignés ; la synchronisation actuelle repose sur une revue statique des sources.

0

erreur consignée au Rebuild

11

essais consignés

3

cas limites documentés

Essai	Résultat consigné	Statut
Détection du point zéro	INIT → nBac = 1 → AUTO	OK
Front du capteur laser	une seule incrémentation pour FALSE → TRUE → FALSE	OK
Consigne atteinte	passage de COMPTAGE à BAC_PLEIN	OK
Quatre cycles du vérin	alternance sortie / rentrée sans défaut intempestif, puis BAC_SUIVANT	OK
Retour 8 vers 1	bouclage logique correct	OK
Bac suivant déjà plein	maintien dans ATTENTE_BAC	OK
Réinitialisation individuelle	seul le compteur visé passe à zéro	OK
Appui maintenu 10 s	remise à zéro des huit compteurs	OK
nPieceBac = 0	passage en DEFAUT, commandes neutralisées	OK
tVerin = T#0MS	passage en DEFAUT, commandes neutralisées	OK
Acquittement après correction	appui 10 s, retour INIT, puis AUTO au zéro	OK

Portée de la présente mise à jour

Les comportements attendus du tableau ont été recoupés avec POU_s/FB_Carrouse1 . TcPOU. La compilation LaTeX du rapport est rejouée, mais le runtime TwinCAT et le Rebuild PLC ne le sont pas dans cette synchronisation documentaire.

08 · RETOUR D'ESSAIS

Cas limites détectés et corrigés

Le relevé d'essais décrit trois comportements limites; leurs corrections sont présentes dans le bloc fonctionnel actuel.

1 • Consigne de pièces égale à zéro

Symptôme	$0 \geq 0$ rendait chaque bac plein immédiatement et lançait une rotation continue.
Correction	contrôle de $nPieceBac \leq 0$ avant l'exécution du cycle AUTO.
Résultat	passage en DEFAUT et neutralisation des deux commandes du vérin.

2 • Huit bacs déjà pleins

Symptôme	après le retour 8 vers 1, le programme pouvait continuer à parcourir les bacs sans intervention opérateur.
Correction	ajout de l'état ATTENTE_BAC lorsque le compteur du nouveau bac a déjà atteint la consigne.
Résultat	arrêt sur le bac concerné, réinitialisation par l'opérateur, puis reprise du comptage.

3 • Temps vérin égal à zéro

Symptôme	le TON terminait immédiatement et les états pouvaient s'enchaîner à la cadence du cycle PLC.
Correction	contrôle de $tVerin \leq T\#0MS$ avec passage en DEFAUT.
Résultat	aucune commande de mouvement tant que le paramètre n'est pas corrigé et acquitté.

Apport principal des essais

Le relevé couvre le cycle nominal, les valeurs limites et la saturation des huit bacs. La revue actuelle confirme la présence des trois corrections dans les sources; elle ne constitue pas un nouveau jeu du runtime.

Limites et évolutions possibles

Distinguer ce qui est couvert par la logique PLC de ce qui dépend encore de la mécanique, de la sécurité ou d’un système industriel complet.

Choix sur le point zéro

L’initialisation attend bZero. Lorsque la référence zéro est détectée par le capteur, nBac prend la valeur 1 et le programme passe en AUTO. Aucune recherche mécanique du zéro n’est inventée, car son mouvement n’est pas défini dans le sujet.

Non implémenté dans cette version

- séquence mécanique complète de prise d’origine;
- fins de course du vérin;
- contrôle réel de position du plateau;
- temporisation de surveillance mécanique indépendante;
- chaîne d’arrêt d’urgence et fonctions de sécurité;
- diagnostic par code défaut détaillé;
- alarmes avancées, acquittements historisés et historique d’événements;

Évolutions proposées

Priorité	Évolution
1	ajouter les capteurs de fin de course et séparer durée de commande et temporisation de surveillance
1	intégrer une autorisation issue de la chaîne de sécurité et définir le comportement à la perte d’énergie
2	mémoriser un code défaut et l’étape d’apparition pour la maintenance, puis ajouter une vue d’alarmes
2	qualifier le filtrage du laser selon la cadence et le capteur réel
2	définir la conservation ou la remise à zéro des compteurs après coupure
3	compléter les IHM ajoutées après l’entretien par un historique, une gestion des droits et des alarmes
3	étudier la variante motorisée avec AXIS_REF et blocs PLCopen Motion

Variante motorisée

MC_Power, MC_MoveModulo, MC_Halt et MC_Stop restent des pistes d’évolution. Aucun de ces blocs n’est présenté comme utilisé ou testé dans la version pneumatique.

Complément hors périmètre initial

Les interfaces réalisées après l’entretien sont documentées séparément en [Annexe B](#). Elles ne sont pas présentées comme une exigence du sujet initial.

Conclusion

Bilan de l'architecture, du relevé d'essais et des limites identifiées.

Résultat obtenu

Le carrousel est structuré autour de trois modes et d'une machine d'états dédiée au cycle AUTO. La logique mémorise les huit compteurs, réalise quatre cycles de commande avant/arrière par indexation et passe en ATTENTE_BAC lorsque le bac suivant est déjà plein.

Résultats des essais

- passage de INIT à AUTO après validation du zéro;
- comptage sur front montant du capteur laser;
- quatre cycles temporisés et bouclage logique du bac 8 vers le bac 1;
- réinitialisation individuelle ou globale des compteurs;
- traitement de trois situations limites;
- acquittement du défaut avec retour par INIT.

Démarche de validation

Le relevé existant couvre le fonctionnement nominal, la saturation des huit bacs et deux paramètres invalides. La synchronisation documentaire a ensuite comparé ces affirmations à MAIN.TcPOU, FB_Carrousel.TcPOU, aux DUTs et au projet .plcproj.

Version livrée

Les sources de référence sont les objets TwinCAT du dépôt : POUs/MAIN.TcPOU, POUs/FB_Carrousel.TcPOU et les deux DUTs. Aucun fichier agrégé FB_Carrousel.st n'est livré.

Synthèse

Le corps du rapport reste centré sur le besoin technique initial. Les deux IHM développées ensuite à titre personnel sont isolées en [Annexe B](#) afin de distinguer clairement le sujet demandé de l'initiative complémentaire.

RÉFÉRENCES TECHNIQUES

- Beckhoff Information System, [R_TRIG - détection d'un front montant dans la bibliothèque Tc2_Standard](#).
- Beckhoff Information System, [exemple TwinCAT 3 associant machine d'états, temporisateur et trigger](#).

ANNEXE A · TWINCAT 3

Types et interface du bloc

Les extraits ci-dessous correspondent aux objets .TcDUT et .TcPOU présents dans le projet.

Énumérations

```
1 {attribute 'qualified_only'}
2 {attribute 'strict'}
3 {attribute 'to_string'}
4 TYPE E_ModeCarrousel :
5 (
6     INIT      := 0,
7     AUTO      := 1,
8     DEFAUT    := 2
9 );
10 END_TYPE
11
12 {attribute 'qualified_only'}
13 {attribute 'strict'}
14 {attribute 'to_string'}
15 TYPE E_EtatCycle :
16 (
17     ATTENTE    := 0,
18     COMPTAGE    := 1,
19     BAC_PLEIN   := 2,
20     VERIN_SORTIE := 3,
21     VERIN_RENTREE := 4,
22     BAC_SUIVANT := 5,
23     ATTENTE_BAC := 6
24 );
25 END_TYPE
```

Lecture en ligne

Les énumérations rendent le mode et l'état courant directement compréhensibles dans TwinCAT, sans devoir interpréter des numéros d'étape.

Interface de FB_Carrousel

```
1 FUNCTION_BLOCK FB_Carrousel
2 VAR_INPUT
3     bLaser      : BOOL; // detection piece
4     bZero       : BOOL; // origine carrousel
5     nPieceBac   : INT;  // consigne / bac
6     tVerin      : TIME; // temps mouvement
7     aResetBac   : ARRAY[1..8] OF BOOL;
8 END_VAR
9
10 VAR_OUTPUT
11     bVerinAv    : BOOL; // cmd avant
12     bVerinAr    : BOOL; // cmd arriere
13     nBac        : INT;  // bac en position
14     bDefaut     : BOOL; // default carrousel
15     eModeHmi    : E_ModeCarrousel;
16     eEtatCycleHmi : E_EtatCycle;
17     aPieceBacHmi : ARRAY[1..8] OF INT;
18     nImpHmi     : INT;
19 END_VAR
20
21 VAR
22     eMode        : E_ModeCarrousel
23                 := E_ModeCarrousel.INIT;
24     eEtatCycle   : E_EtatCycle
25                 := E_EtatCycle.ATTENTE;
26     aPieceBac    : ARRAY[1..8] OF INT;
27     nImp         : INT := 0;
28     fbLaser      : R_TRIG;
29     fbTempoVerin : TON;
30     aTempoReset  : ARRAY[1..8] OF TON;
31     bResetGlobal : BOOL;
32     i            : INT;
33     j            : INT;
34 END_VAR
```

Variables exposées à la Visualisation

MAIN relie les quatre sorties HMI du bloc à eModeOut, eEtatCycleOut, aPieceBacOut et nImpOut. Il dérive aussi sModeVisu, sEtatCycleVisu et aBacActifVisu[1..8] pour l'affichage.

ANNEXE B · INITIATIVE PERSONNELLE

IHM ajoutées après l'entretien

Deux niveaux de supervision ont été développés en complément du sujet initial : une PLC Visualization V1, puis une application TwinCAT HMI V2.

Positionnement du complément

L'interface opérateur ne faisait pas partie de la demande formulée pendant l'entretien. Elle a été ajoutée ensuite à titre personnel pour prolonger la démarche : rendre le cycle observable, faciliter la simulation et montrer une séparation claire entre logique PLC, communication et présentation. Cette annexe documente ce travail sans l'intégrer artificiellement au périmètre initial.

B.1 · PLC Visualization V1

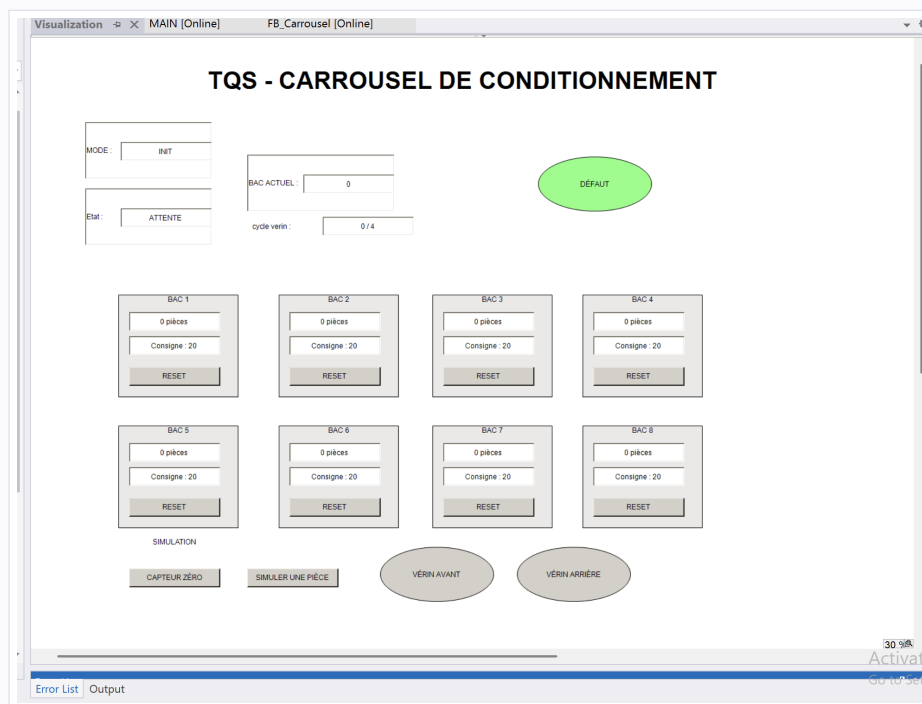


FIGURE 1 – PLC Visualization V1 ouverte en ligne dans TwinCAT XAE.

Nature de l'interface

Cette première version est intégrée au projet automate. Visualization.TcVIS, son gestionnaire et la liste de textes constituent une vue locale destinée à la mise au point et à la simulation dans TwinCAT XAE.

Informations et commandes exposées

Mode, état du cycle, bac actif, compteurs des huit bacs, consigne, progression des quatre cycles vérin, défaut et commandes avant/arrière. Les boutons permettent de simuler le capteur zéro, le passage d'une pièce et la réinitialisation des bacs.

ANNEXE B · SUITE

TwinCAT HMI V2

Une seconde interface web reprend les données du PLC dans un tableau de bord plus lisible et sépare explicitement la communication ADS de l’affichage.

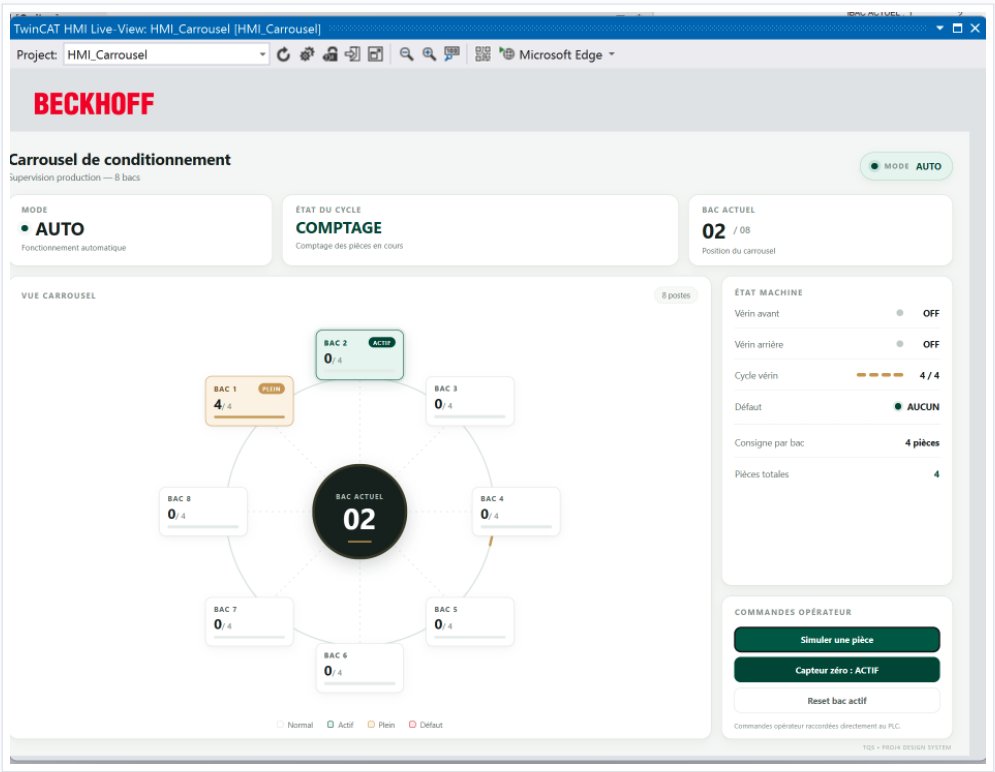


FIGURE 2 – TwinCAT HMI V2 dans la Live-View : bac 2 actif, bac 1 plein et cycle de comptage en cours.

Architecture retenue

PlcService encapsule la lecture, la surveillance et l’écriture des symboles ADS. CarouselController coordonne les échanges. DashboardView met à jour l’écran et DashboardTemplate construit sa structure. Le fichier source Themes/Base/TqsDashboard.css centralise le thème graphique, les couleurs d’état et la mise en page responsive. La logique machine reste exclusivement dans le PLC.

Échanges avec le PLC

Sens	Données
PLC vers IHM	mode, état, bac actif, consigne, huit compteurs, cycle vérin, commandes avant/arrière et défaut
IHM vers PLC	simulation pièce, capteur zéro et reset momentané du bac actif

Limite conservée

Cette interface complète la démonstration, mais ne remplace pas une IHM de production qualifiée : les alarmes historisées, la gestion des droits, la sécurité machine et le diagnostic mécanique restent hors périmètre.